

BLE MIOTY BLE TX - `main.c` and `main.h` Explained

A layered, beginner-friendly walkthrough of the STM32/SX1280 BLE advertising transmitter firmware.

Source project explained:

`STM Codes/2_ble_mioty_ble_tx_ble_working`

Files explained:

- `Core/Src/main.c`
- `Core/Inc/main.h`

This explanation is written from the wide view first, then it slowly zooms into the code flow one part at a time.

1. Very Wide Overview

This firmware runs on an STM32F1 microcontroller. Its job is to control an SX1280 radio chip and make that radio transmit BLE advertising packets.

In simple words:

1. The STM32 starts up.
2. It configures its clock, GPIO pins, and SPI peripheral.
3. It resets the SX1280 radio chip.
4. It checks that the radio answers over SPI.
5. It configures the SX1280 to behave like a BLE advertiser transmitter.
6. It builds an iBeacon-style BLE advertising packet.
7. It sends that packet on the three BLE advertising channels:
 - Channel 37: 2402 MHz
 - Channel 38: 2426 MHz
 - Channel 39: 2480 MHz
8. It repeats forever.

The important idea is this:

The STM32 is not using a normal BLE stack. It is manually building a BLE advertisement packet and asking the SX1280 radio to transmit it.

That makes the code low-level. You see many direct radio commands such as `0x8A`, `0x8B`, `0x83`, and many byte arrays. Those bytes are not random. They are commands and settings from the SX1280 radio command interface.

2. What Each File Does

main.h

main.h is the shared header for main.c.

It mainly does three things:

1. Includes the STM32 HAL library.
2. Declares `Error_Handler()`.
3. Gives friendly names to hardware pins.

Example:

```
#define LORA_NSS_Pin GPIO_PIN_4
#define LORA_NSS_GPIO_Port GPIOA
```

This means:

The SX1280 chip select pin, called `LORA_NSS`, is connected to pin PA4 on the STM32.

main.c

main.c is the main firmware file.

It contains:

- The program entry point: `main()`
- STM32 setup functions:
 - `SystemClock_Config()`
 - `MX_GPIO_Init()`
 - `MX_SPI1_Init()`
- SX1280 helper functions:
 - send SPI commands
 - read and write registers
 - wait for the radio to become ready
- BLE radio setup:
 - configure SX1280 packet type as BLE
 - configure frequency, packet parameters, CRC, access address, whitening seed
- The endless transmit loop.

3. Hardware Cast of Characters

STM32F1

This is the microcontroller running the C code.

It controls pins, runs the main loop, and sends commands over SPI.

SX1280

This is the 2.4 GHz radio chip.

The code uses it to transmit BLE advertising packets.

SPI

SPI is the communication bus between the STM32 and SX1280.

Think of SPI like a very simple conversation:

- STM32 is the master.
- SX1280 is the slave.
- STM32 pulls chip select low.
- STM32 sends bytes.
- SX1280 receives those bytes as commands or data.
- STM32 releases chip select high.

In this code, SPI1 is used.

GPIO

GPIO means general-purpose input/output pins.

These are normal digital pins. The code uses them for:

- LED output
- SX1280 chip select
- SX1280 reset
- SX1280 busy input
- SX1280 DIO1 interrupt/status input

4. Pin Map From `main.h`

The project defines the important pins like this:

Name	STM32 pin	Direction	Meaning
LED_Pin	PC13	Output	Debug/status LED
LORA_NSS_Pin	PA4	Output	SX1280 SPI chip select, active low
LORA_NRST_Pin	PA3	Output	SX1280 reset pin, active low
LORA_DIO1_Pin	PB0	Input	SX1280 DIO1 pin, used here for TxDone
LORA_BUSY_Pin	PB15	Input	SX1280 busy pin

Important small detail:

NSS is active low. That means:

- `GPIO_PIN_RESET` selects the radio.
- `GPIO_PIN_SET` deselects the radio.

So when the code sends an SPI command, it does this pattern:

```
HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_RESET);  
// send bytes over SPI  
HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
```

That means:

1. Start talking to the radio.
2. Send command/data.
3. Stop talking to the radio.

5. Basic C Concepts Used Here

Before dissecting the firmware flow, here are the main C basics used in the code.

`uint8_t`, `uint16_t`, `uint32_t`

These are fixed-size unsigned integer types.

Type	Size	Range	Used for
<code>uint8_t</code>	8 bits, 1 byte	0 to 255	command bytes, registers, packet bytes
<code>uint16_t</code>	16 bits, 2 bytes	0 to 65535	register addresses
<code>uint32_t</code>	32 bits, 4 bytes	0 to about 4 billion	timestamps, counters

Radio commands are byte-oriented, so the code uses `uint8_t` a lot.

Arrays

An array is a row of values.

Example:

```
uint8_t cmd[4] = {0x86, b0, b1, b2};
```

This creates four bytes. The first byte is the command opcode `0x86`, and the next three bytes are the frequency bytes.

Functions

A function is a named block of code.

Example:

```
void SX1280_SetFrequency(uint8_t b0, uint8_t b1, uint8_t b2)
```

This function sends a frequency-setting command to the SX1280.

`void`

`void` means the function does not return a value.

Example:

```
void LED_Blink(uint8_t count)
```

This function blinks the LED, but it does not return anything.

`while (1)`

This means "loop forever".

Embedded firmware often has one forever loop because the device is meant to keep running until power is removed or reset.

`HAL_...`

Functions beginning with `HAL_` come from ST's Hardware Abstraction Layer.

HAL lets the code control STM32 hardware without directly writing every register manually.

Examples:

- `HAL_GPIO_WritePin()` sets a pin high or low.
- `HAL_GPIO_ReadPin()` reads a pin.
- `HAL_SPI_Transmit()` sends SPI bytes.
- `HAL_Delay()` waits for a number of milliseconds.
- `HAL_GetTick()` gives a millisecond counter.

6. Full Firmware Story in Plain English

Here is the whole boot-to-transmit story:

```
Power on / reset
|
v
STM32 startup code calls main()
|
v
Small delay so ST-Link can reconnect
|
v
Initialize STM32 HAL
|
v
Configure system clock
|
v
Configure GPIO pins
|
v
Configure SPI1
|
```

```

      v
Reset SX1280 radio
      |
      v
Ask SX1280 for status
      |
      +--> bad response: blink slowly forever
      |
      +--> good response: blink once
      |
      v
Configure SX1280 for BLE advertising
      |
      v
Read back important settings and blink diagnostic pattern
      |
      v
Forever:
    send BLE advertisement on channel 37
    send BLE advertisement on channel 38
    send BLE advertisement on channel 39
    blink/toggle LED
    wait a little

```

7. Top of main.c

The file starts with:

```
#include "main.h"
```

This brings in:

- STM32 HAL definitions
- pin names from `main.h`
- `Error_Handler()` declaration

Then:

```
SPI_HandleTypeDef hspi1;
```

This creates a handle for SPI1.

Think of `hspi1` as a configuration/control object for SPI1. Whenever the code calls:

```
HAL_SPI_Transmit(&hspi1, ...)
```

it is saying:

"Use SPI1 to transmit these bytes."

8. Function Prototypes

Near the top, the code lists function prototypes:

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI1_Init(void);
```

and:

```
void SX1280_InitBLE(void);
```

```
void SX1280_SendBLEBeacon(void);
```

A prototype tells the compiler:

"This function exists later in the file. You can allow calls to it before you have seen its full body."

Without prototypes, C can complain when one function calls another function that is defined later.

9. LED_Blink()

Code idea:

```
void LED_Blink(uint8_t count) {
    for (uint8_t i = 0; i < count; i++) {
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
        HAL_Delay(150);
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
        HAL_Delay(150);
    }
    HAL_Delay(600);
}
```

Plain English:

Blink the LED `count` times.

Each blink:

1. Turn LED on.
2. Wait 150 ms.
3. Turn LED off.
4. Wait 150 ms.

After all blinks, wait 600 ms.

Why this helper exists:

The firmware has no screen, serial console, or debugger output here. So it uses LED blink patterns as messages.

Examples:

- 1 blink means the SX1280 answered over SPI.
- 9, 6, 3, 1 blink groups mean the BLE setup register readback looked correct.
- Fast blinking forever means an error condition.

10. SX1280_WaitBusy()

The SX1280 has a `BUSY` pin.

When `BUSY` is high, the chip is busy and should not receive a new command.

When `BUSY` is low, the chip is ready.

Code idea:

```

while (HAL_GPIO_ReadPin(LORA_BUSY_GPIO_Port, LORA_BUSY_Pin) == GPIO_PIN_SET) {
    if ((HAL_GetTick() - timeout) > 1000) {
        while(1) {
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
            HAL_Delay(50);
        }
    }
}

```

Plain English:

1. Remember the current time.
2. Keep checking the BUSY pin.
3. If BUSY goes low, continue.
4. If BUSY stays high for more than 1 second, something is wrong.
5. If something is wrong, blink the LED very fast forever.

Why this is important:

The radio command interface depends on timing. If the STM32 sends a command while the radio is still busy, the command may be ignored or corrupted.

11. SX1280_GetStatus()

This function asks the SX1280:

"What state are you in?"

It sends:

```
uint8_t tx[2] = {0xC0, 0x00};
```

0xC0 is the SX1280 GetStatus command.

The function then receives two bytes and returns rx[1].

The status byte contains fields:

Bits	Meaning
7:5	circuit mode
4:2	command status

The comments say examples for circuit mode:

- 2 = standby using RC oscillator
- 3 = standby using crystal oscillator
- 4 = frequency synthesis
- 5 = receive
- 6 = transmit

The code later uses this status for startup checking and diagnostic LED blink patterns.

12. SX1280_SendCommand()

This is the basic "send bytes to radio" helper.

Code idea:

```
void SX1280_SendCommand(uint8_t *cmd, uint8_t len) {
    SX1280_WaitBusy();
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, cmd, len, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
}
```

Plain English:

1. Wait until the radio is ready.
2. Pull chip select low.
3. Send `len` bytes from the `cmd` array over SPI.
4. Pull chip select high.

Almost every SX1280 command goes through this helper.

Important detail:

`HAL_MAX_DELAY` means the SPI transmit call waits as long as needed. If SPI gets stuck, the firmware can block there forever.

13. SX1280_WriteBuffer()

This writes packet bytes into the SX1280's internal transmit buffer.

Code idea:

```
uint8_t header[2] = {0x1A, offset};
HAL_SPI_Transmit(&hspi1, header, 2, HAL_MAX_DELAY);
HAL_SPI_Transmit(&hspi1, data, len, HAL_MAX_DELAY);
```

Plain English:

1. Send opcode `0x1A`, which means WriteBuffer.
2. Send the starting buffer offset.
3. Send the actual packet bytes.

In this firmware, the packet is written at offset `0x80` because the code configures the TX base address to `0x80`.

Those two things must agree:

```
SetBufferBaseAddresses TX base = 0x80
WriteBuffer offset = 0x80
```

If they do not agree, the radio may transmit from the wrong memory area.

14. SX1280_WriteRegister()

This writes bytes into SX1280 registers.

Code idea:

```
uint8_t header[3] = {0x18, (addr >> 8) & 0xFF, addr & 0xFF};
```

Plain English:

1. 0x18 is the SX1280 WriteRegister opcode.
2. The register address is 16 bits, so it is split into two bytes:
 - high byte
 - low byte
3. Then the code sends the data bytes.

Example:

```
SX1280_WriteRegister(0x09CF, access_addr, 4);
```

This writes four bytes starting at register 0x09CF.

15. SX1280_ReadRegister()

This reads one byte from an SX1280 register.

Code idea:

```
uint8_t header[4] = {0x19, addrHigh, addrLow, 0x00};  
HAL_SPI_Transmit(&hspi1, header, 4, HAL_MAX_DELAY);  
HAL_SPI_Receive(&hspi1, &val, 1, HAL_MAX_DELAY);
```

Plain English:

1. Send opcode 0x19, which means ReadRegister.
2. Send the 16-bit register address.
3. Send one dummy byte.
4. Receive one real byte.

The dummy byte is common in SPI read protocols. SPI always clocks data in and out at the same time, so sometimes you must transmit dummy bytes just to generate clock pulses for receiving.

16. SX1280_SetFrequency()

This function sends the SX1280 SetRfFrequency command.

Code idea:

```
uint8_t cmd[4] = {0x86, b0, b1, b2};  
SX1280_SendCommand(cmd, 4);
```

Plain English:

1. 0x86 means SetRfFrequency.

2. The next three bytes select the RF frequency.

The code already has known-good values:

BLE channel	Frequency	Bytes
37	2402 MHz	0xB8 0xC4 0xEC
38	2426 MHz	0xBA 0x9D 0x89
39	2480 MHz	0xBE 0xC4 0xEC

These are the three official BLE advertising channels.

17. SX1280_InitBLE()

This is one of the most important functions.

It prepares the SX1280 for BLE advertising packet transmission.

It is called once after reset, before any packet is transmitted.

Step 1: Standby mode

```
uint8_t cmd_standby[2] = {0x80, 0x00};
SX1280_SendCommand(cmd_standby, 2);
```

Meaning:

Tell the radio to enter standby mode using the RC oscillator.

The radio should be in a known, calm state before configuring packet settings.

Step 2: Packet type = BLE

```
uint8_t cmd_pkt_type[2] = {0x8A, 0x04};
```

Meaning:

0x8A is SetPacketType.

0x04 selects BLE packet mode.

This tells the SX1280:

"Interpret packet settings as BLE packet settings."

Step 3: Buffer base addresses

```
uint8_t cmd_buf[3] = {0x8F, 0x80, 0x00};
```

Meaning:

0x8F is SetBufferBaseAddresses.

- TX base address = 0x80
- RX base address = 0x00

This firmware only transmits, so the TX base matters most.

Later the code writes the packet to offset 0x80.

Step 4: Modulation parameters

```
uint8_t cmd_mod[4] = {0x8B, 0x45, 0x01, 0x20};
```

Meaning:

Configure BLE-like radio modulation:

- 1 Mbps
- modulation index 0.5
- BT 0.5

In simple words:

This controls how the bits are physically shaped and transmitted over the air.

Step 5: TX output power

```
uint8_t cmd_tx_params[3] = {0x8E, 0x1F, 0x20};
```

Meaning:

Set transmit power and ramp time.

The comment says:

- output power = +13 dBm
- ramp time = 20 us

Transmit power affects range and current consumption.

Step 6: BLE advertising access address

```
uint8_t access_addr[4] = {0x8E, 0x89, 0xBE, 0xD6};
SX1280_WriteRegister(0x09CF, access_addr, 4);
```

BLE advertising packets use a fixed access address:

```
0x8E89BED6
```

The code writes that value into SX1280 registers starting at 0x09CF.

This is required so BLE scanners can recognize the signal as BLE advertising traffic.

Step 7: BLE advertising CRC seed

```
uint8_t crc_seed[3] = {0x55, 0x55, 0x55};
```

```
SX1280_WriteRegister(0x09C7, crc_seed, 3);
```

BLE advertising uses CRC init:

```
0x555555
```

CRC is an error-detection value. Receivers use it to check whether the packet was corrupted.

Step 8: Map TxDone IRQ to DIO1

```
uint8_t cmd_irq[9] = {0x8D, 0x00, 0x01, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00};
```

Meaning:

Tell the SX1280 to raise the DIO1 pin when transmission is done.

Later, after starting TX, the code checks:

```
HAL_GPIO_ReadPin(LORA_DIO1_GPIO_Port, LORA_DIO1_Pin)
```

If DIO1 is high, the code treats that as TxDone confirmed.

18. SX1280_VerifyInit()

This function is a startup diagnostic.

It reads back important radio values and turns them into LED blink groups.

Expected pattern:

```
9 blinks
6 blinks
3 blinks
1 blink
```

Why those numbers?

Group 1: Access address high nibble

```
uint8_t aa = SX1280_ReadRegister(0x09CF);
LED_Blink((aa >> 4) + 1);
```

Register `0x09CF` should contain `0x8E`.

The high nibble of `0x8E` is `0x8`.

`8 + 1 = 9`.

So expected group 1 is 9 blinks.

Group 2: CRC seed high nibble

```
uint8_t crc = SX1280_ReadRegister(0x09C7);
LED_Blink((crc >> 4) + 1);
```

Register `0x09C7` should contain `0x55`.

The high nibble of 0x55 is 0x5.

5 + 1 = 6.

So expected group 2 is 6 blinks.

Group 3: circuit mode

```
uint8_t st = SX1280_GetStatus();
uint8_t circuit_mode = (st >> 5) & 0x07;
LED_Blink(circuit_mode + 1);
```

Expected circuit mode is standby RC, value 2.

2 + 1 = 3.

So expected group 3 is 3 blinks.

Group 4: command status

```
uint8_t cmd_status = (st >> 2) & 0x07;
LED_Blink(cmd_status + 1);
```

Expected command status is idle, value 0.

0 + 1 = 1.

So expected group 4 is 1 blink.

Why add 1?

The code adds 1 so that value 0 is still visible as 1 blink.

If it did not add 1, a value of 0 would mean no blinks, which is hard to distinguish from a broken LED or missed observation.

19. SX1280_SendOnChannel()

This function transmits one BLE advertising packet on one channel.

Function signature:

```
uint8_t SX1280_SendOnChannel(uint8_t freq0, uint8_t freq1, uint8_t freq2,
                             uint8_t ch_index, uint8_t white_seed)
```

Inputs:

Parameter	Meaning
freq0, freq1, freq2	three SX1280 frequency bytes
ch_index	BLE channel number, currently not used inside the function
white_seed	whitening seed for this BLE channel

Return value:

Return	Meaning
1	DIO1 showed TxDone
0	TxDone was not seen

Step 1: Set frequency

```
SX1280_SetFrequency(freq0, freq1, freq2);
```

The SX1280 is tuned to the requested BLE advertising channel.

Step 2: Re-send modulation parameters

```
uint8_t cmd_mod[4] = {0x8B, 0x45, 0x01, 0x20};  
SX1280_SendCommand(cmd_mod, 4);
```

This repeats the BLE modulation setup.

This may be done to make sure the radio still has the right modulation settings before each transmission.

Step 3: Build BLE advertising data

The code builds this:

```
uint8_t adv_data[] = {  
    0x02, 0x01, 0x06,  
    0x1A, 0xFF,  
    0x4C, 0x00,  
    0x02, 0x15,  
    0x01, 0x02, 0x03, 0x04,  
    0x05, 0x06, 0x07, 0x08,  
    0x09, 0x0A, 0x0B, 0x0C,  
    0x0D, 0x0E, 0x0F, 0x10,  
    0x00, 0x01,  
    0x00, 0x02,  
    0xC5  
};
```

This is the BLE advertisement payload data.

It contains two advertising data structures.

Advertising data structure 1: flags

```
0x02 0x01 0x06
```

Breakdown:

Byte	Meaning
0x02	length: 2 bytes after this
0x01	AD type: flags
0x06	flag value: general discoverable, BR/EDR not supported

Simple meaning:

"This is a BLE-only discoverable advertisement."

Advertising data structure 2: iBeacon manufacturer data

```
0x1A 0xFF 0x4C 0x00 0x02 0x15 ...
```

Breakdown:

Bytes	Meaning
0x1A	length: 26 bytes after this
0xFF	AD type: manufacturer specific data
0x4C 0x00	Apple company ID, little-endian for 0x004C
0x02	iBeacon subtype
0x15	iBeacon data length, 21 bytes
next 16 bytes	proximity UUID
next 2 bytes	major value
next 2 bytes	minor value
0xC5	measured TX power at 1 m

The UUID used in this code is:

```
01020304-0506-0708-090A-0B0C0D0E0F10
```

The iBeacon major value is:

```
1
```

The iBeacon minor value is:

```
2
```

The measured power byte is:

```
0xC5
```

As a signed 8-bit value, 0xC5 means -59.

So this advertises measured power of about -59 dBm at 1 meter.

Step 4: Set advertiser address

```
uint8_t mac[6] = {0xFF, 0xEE, 0xDD, 0xCC, 0xBB, 0xAA};
```

The code stores the BLE advertiser address in least-significant-byte-first order.

The comment says that on air it appears as:

```
FF:EE:DD:CC:BB:AA
```

BLE byte order is easy to get confused by. The code comment is there because the byte order in memory and the human-readable address order can look reversed depending on the tool.

The previous `1_ble_mioty_ble_tx_working` version used a different address:

```
C1:05:06:03:02:01
```

The current `2_ble_mioty_ble_tx_ble_working` version uses:

```
FF:EE:DD:CC:BB:AA
```

That is the only difference found between the `1_` and `2_` `main.c` files.

Step 5: Build full BLE PDU

The code says:

```
uint8_t pdu_len = 6 + 30;
uint8_t payload[38];
```

Why:

- BLE advertiser address = 6 bytes
- advertising data = 30 bytes
- total PDU body = 36 bytes
- BLE PDU header = 2 bytes
- total packet written to SX1280 buffer = 38 bytes

Then:

```
payload[0] = 0x42;
payload[1] = pdu_len;
```

First byte `0x42` is the BLE advertising PDU header byte.

In simple terms:

- lower bits select `ADV_NONCONN_IND`
- bit 6 says the advertiser address is random

`ADV_NONCONN_IND` means:

"This is a non-connectable advertisement."

That means scanners can see it, but devices are not meant to connect to it.

Second byte is length:

```
36
```

Then the code copies:

1. 6 bytes of advertiser address into `payload[2]` through `payload[7]`.
2. 30 bytes of advertising data into `payload[8]` through `payload[37]`.

Step 6: Set BLE packet parameters

```
uint8_t cmd_pkt[8] = {0x8C, 0x20, 0x10, 0x04, 0x00, 0x00, 0x00, 0x00};
SX1280_SendCommand(cmd_pkt, 8);
```

Meaning:

Send the SX1280 SetPacketParams command for BLE mode.

The comments say this matches the reference board's `prepareForBeacon()` function.

Important detail from the code:

Whitening is not enabled by that packet parameter byte. Instead, the whitening seed is written directly to register `0x09C5`.

Step 7: Write whitening seed

```
SX1280_WriteRegister(0x09C5, &white_seed, 1);
```

Whitening is a BLE process that scrambles the transmitted bits in a predictable way. This prevents long runs of zeros or ones and helps RF performance.

Each BLE advertising channel uses a different whitening seed:

BLE channel	Seed
37	0x53
38	0x33
39	0x73

Step 8: Write packet to SX1280 TX buffer

```
SX1280_WriteBuffer(0x80, payload, 38);
```

Meaning:

Copy the full 38-byte packet into the SX1280 transmit buffer at offset `0x80`.

Again, this offset matches the TX buffer base configured during `SX1280_InitBLE()`.

Step 9: Start transmission

```
uint8_t cmd_tx[4] = {0x83, 0x00, 0x00, 0x64};  
SX1280_SendCommand(cmd_tx, 4);
```

`0x83` is the SetTx command.

The comments say:

- period base = `0x00`
- count = `0x0064`
- timeout is around 1.56 ms

Simple meaning:

"Radio, transmit the packet now."

Step 10: Wait and check TxDone

```
SX1280_WaitBusy();
HAL_Delay(2);
```

The code waits for the radio's BUSY pin to go low again.

Then it checks DIO1:

```
if (HAL_GPIO_ReadPin(LORA_DIO1_GPIO_Port, LORA_DIO1_Pin) == GPIO_PIN_SET) {
    txdone = 1;
    uint8_t clr[3] = {0x97, 0xFF, 0xFF};
    SX1280_SendCommand(clr, 3);
}
```

If DIO1 is high:

1. The code marks TxDone as true.
2. It sends ClearIrqStatus command 0x97.
3. It clears all IRQ flags with 0xFFFF.

Clearing IRQ flags is important because otherwise DIO1 might stay high and confuse the next transmission.

Finally:

```
return txdone;
```

The function returns whether TxDone was observed.

20. SX1280_SendBLEBeacon()

This function sends one beacon on all three BLE advertising channels.

Code:

```
SX1280_SendOnChannel(0xB8, 0xC4, 0xEC, 37, 0x53);
HAL_Delay(10);
SX1280_SendOnChannel(0xBA, 0x9D, 0x89, 38, 0x33);
HAL_Delay(10);
SX1280_SendOnChannel(0xBE, 0xC4, 0xEC, 39, 0x73);
HAL_Delay(10);
```

Plain English:

1. Transmit on channel 37.
2. Wait 10 ms.
3. Transmit on channel 38.
4. Wait 10 ms.
5. Transmit on channel 39.
6. Wait 10 ms.

Why all three channels?

BLE advertisements are normally sent on channels 37, 38, and 39 so scanners have a better chance of hearing them.

Important observation:

`SX1280_SendOnChannel()` returns 1 or 0, but `SX1280_SendBLEBeacon()` currently ignores those return values.

So even though each channel can report TxDone, the main loop does not currently use that information.

21. `main()`

This is the central flow of the firmware.

Step 1: ST-Link reconnect delay

```
for (volatile uint32_t i = 0; i < 5000000; i++);
```

This is a crude startup delay.

The comment says it gives ST-Link time to attach after flashing.

The word `volatile` matters here. It tells the compiler:

"Do not optimize this loop away."

Without `volatile`, the compiler might decide the loop does nothing useful and remove it.

Step 2: HAL init

```
HAL_Init();
```

This initializes the STM32 HAL layer.

It sets up core HAL features such as the system tick timer used by `HAL_Delay()` and `HAL_GetTick()`.

Step 3: System clock

```
SystemClock_Config();
```

This configures the STM32 clock tree.

The code uses:

- external high-speed oscillator: HSE
- PLL source: HSE
- PLL multiplier: 9

On a typical STM32F103 board with an 8 MHz HSE crystal:

$$8 \text{ MHz} * 9 = 72 \text{ MHz system clock}$$

The code also sets:

- AHB divider = 1
- APB1 divider = 2
- APB2 divider = 1

So a common result is:

- CPU / HCLK = 72 MHz
- APB1 = 36 MHz
- APB2 = 72 MHz

SPI1 is on APB2, so its clock source is related to APB2.

Step 4: GPIO init

```
MX_GPIO_Init();
```

This configures pins for:

- LED
- SX1280 NSS
- SX1280 reset
- SX1280 BUSY
- SX1280 DIO1

Step 5: SPI init

```
MX_SPI1_Init();
```

This configures SPI1 as the master interface to the SX1280.

Step 6: Deselect radio

```
HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
```

This makes sure the SX1280 is not selected before the reset/configuration sequence begins.

Step 7: Hard reset SX1280

```
HAL_GPIO_WritePin(LORA_NRST_GPIO_Port, LORA_NRST_Pin, GPIO_PIN_RESET);  
HAL_Delay(5);  
HAL_GPIO_WritePin(LORA_NRST_GPIO_Port, LORA_NRST_Pin, GPIO_PIN_SET);  
HAL_Delay(10);
```

Plain English:

1. Pull reset low.
2. Wait 5 ms.
3. Release reset high.
4. Wait 10 ms for the radio to boot.

This puts the SX1280 in a known fresh state.

Step 8: Check SX1280 SPI response

```
uint8_t status = SX1280_GetStatus();  
if (status == 0x00 || status == 0xFF) {
```

```

        while(1) {
            HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
            HAL_Delay(1000);
        }
    }
}

```

The code asks the radio for its status.

If the response is `0x00` or `0xFF`, the code treats that as a bad SPI response.

Why?

- `0x00` can mean the line is stuck low or no useful response came back.
- `0xFF` can mean the line is stuck high, floating, or the chip is not responding.

If the response is bad, the firmware slow-blinks forever.

This is a startup failure indicator.

Step 9: Alive blink

```
LED_Blink(1);
```

If the SX1280 responded, the firmware blinks once.

Plain meaning:

"The radio is alive enough to answer over SPI."

Step 10: BLE radio setup

```
SX1280_InitBLE();
```

This configures the radio for BLE packet transmission.

Step 11: Diagnostic readback

```
SX1280_VerifyInit();
```

This reads back important registers and blinks the expected `9, 6, 3, 1` pattern.

This helps answer:

"Did the firmware successfully write the important BLE settings into the radio?"

Step 12: Forever transmit loop

```

while (1)
{
    SX1280_SendBLEBeacon();
    HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
    HAL_Delay(20);
    HAL_Delay(200);
}

```

Plain English:

Forever:

1. Send a BLE beacon on channels 37, 38, and 39.
2. Toggle the LED.
3. Wait 20 ms.
4. Wait another 200 ms.
5. Repeat.

There is a large commented-out section that refers to `ch37_ok`, `ch38_ok`, and `ch39_ok`.

Those variables are currently not active. That means the current loop does not actually display per-channel TxDone success/failure.

The comments describe a more advanced LED pattern, but the active code only toggles the LED briefly after sending.

22. `SystemClock_Config()`

This function configures the STM32 clock.

Important settings:

```
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
RCC_OscInitStruct.HSEState = RCC_HSE_ON;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
```

Plain English:

Use the external oscillator, feed it into the PLL, and multiply it by 9.

Then:

```
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
```

Plain English:

Use the PLL output as the main system clock.

If clock setup fails, the code calls:

```
Error_Handler();
```

23. `MX_SPI1_Init()`

This configures SPI1.

Settings:

Setting	Value	Meaning
Mode	master	STM32 controls SPI clock

Setting	Value	Meaning
Direction	2 lines	separate MOSI and MISO
DataSize	8-bit	send bytes
CLKPolarity	low	clock idles low
CLKPhase	first edge	sample on first clock edge
NSS	software	code manually controls chip select
BaudRatePrescaler	8	SPI clock divided by 8
FirstBit	MSB first	send most significant bit first
TIMode	disabled	normal SPI, not TI mode
CRCCalculation	disabled	no STM32 SPI CRC

Together, CPOL low and CPHA first edge mean SPI mode 0.

If APB2 is 72 MHz, prescaler 8 gives an SPI clock around:

$$72 \text{ MHz} / 8 = 9 \text{ MHz}$$

That is the approximate SPI speed, assuming the typical 72 MHz APB2 setup.

24. MX_GPIO_Init()

This configures the pins.

Enable port clocks

```
__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOD_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();
```

Before using GPIO pins, the STM32 must enable the clock for each GPIO port.

No GPIO port clock means the port does not respond properly to configuration.

Preload output values

```
HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
HAL_GPIO_WritePin(LORA_NSS_GPIO_Port, LORA_NSS_Pin, GPIO_PIN_SET);
HAL_GPIO_WritePin(LORA_Nrst_GPIO_Port, LORA_Nrst_Pin, GPIO_PIN_SET);
```

This sets desired output levels before configuring the pins as outputs.

Why?

It reduces glitches.

For example, LORA_NSS is set high so the radio is not accidentally selected during startup.

Configure LED

```
GPIO_InitStruct.Pin = LED_Pin;
```

```

GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(LED_GPIO_Port, &GPIO_InitStruct);

```

This makes PC13 a push-pull output.

Push-pull means the pin can actively drive high and low.

Configure SX1280 NSS

```

GPIO_InitStruct.Pin = LORA_NSS_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
HAL_GPIO_Init(LORA_NSS_GPIO_Port, &GPIO_InitStruct);

```

This makes PA4 an output for chip select.

The code manually drives it low/high around each SPI transaction.

Configure SX1280 reset

```

GPIO_InitStruct.Pin = LORA_NRST_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
HAL_GPIO_Init(LORA_NRST_GPIO_Port, &GPIO_InitStruct);

```

This makes PA3 an output for the radio reset line.

Configure BUSY and DIO1

```

GPIO_InitStruct.Pin = LORA_DIO1_Pin | LORA_BUSY_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

```

This configures PB0 and PB15 as inputs.

- PB15 reads SX1280 BUSY.
- PB0 reads SX1280 DIO1.

No internal pull resistor is enabled, so the external hardware must drive these lines clearly.

25. Error_Handler()

Code:

```

void Error_Handler(void)
{
    while (1)
    {
        HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);
        HAL_Delay(100);
    }
}

```

Plain English:

If a serious STM32 setup error happens, blink the LED quickly forever.

This is used if:

- clock configuration fails
- SPI initialization fails

The code does not try to recover. In embedded bring-up code, this is common because a clock or peripheral setup failure usually means the firmware cannot continue safely.

26. The BLE Packet Built by the Code

The final packet written to the SX1280 TX buffer is 38 bytes:

```
2 bytes BLE PDU header
6 bytes advertiser address
30 bytes advertising data
```

Expanded:

```
Header:
  0x42
  0x24

Address:
  FF EE DD CC BB AA

Advertising data:
  02 01 06
  1A FF 4C 00 02 15
  01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10
  00 01
  00 02
  C5
```

Note:

0x24 is decimal 36.

That length is:

```
6 address bytes + 30 advertising data bytes = 36
```

So the actual payload array is:

```
42 24 FF EE DD CC BB AA 02 01 06 1A FF 4C 00 02 15
01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10
00 01 00 02 C5
```

27. Why Three Different Whitening Seeds?

BLE uses data whitening. Each channel uses a channel-dependent seed.

This code uses:

```
Channel 37 -> 0x53
Channel 38 -> 0x33
Channel 39 -> 0x73
```

If the whitening seed is wrong, the radio may still transmit energy, but a BLE scanner may not decode the packet correctly.

So the seed is not just a small detail. It is part of making the packet look valid to BLE receivers.

28. LED Patterns and What They Mean

Fast 50 ms blink forever

Where:

`SX1280_WaitBusy()`

Meaning:

The SX1280 BUSY pin stayed high for more than 1 second.

Possible causes:

- radio not powered
- BUSY pin wiring issue
- wrong pin mapping
- radio stuck
- command/timing problem

Slow 1 second blink forever

Where:

startup SPI status check in `main()`

Meaning:

The SX1280 returned `0x00` or `0xFF` for status.

Possible causes:

- SPI wiring issue
- NSS wiring issue
- MISO stuck high/low
- SX1280 not powered
- wrong SPI mode
- reset line problem

One blink after startup

Where:

after `SX1280_GetStatus()` passes

Meaning:

The SX1280 responded over SPI.

9, 6, 3, 1 diagnostic pattern

Where:

`SX1280_VerifyInit()`

Meaning:

The key BLE setup values read back correctly.

If this pattern is correct but BLE scanning still fails, the problem may be in:

- RF path
- antenna
- BLE packet formatting
- whitening
- frequency settings
- scanner expectations
- transmit power or range

29. Important Details and Gotchas

`ch_index` is unused

`SX1280_SendOnChannel()` receives `ch_index`, but the function does not use it.

That is not harmful, but it means the channel number is only there for readability or future debugging.

TxDone result is ignored by the caller

`SX1280_SendOnChannel()` returns whether DIO1 showed TxDone.

But `SX1280_SendBLEBeacon()` does not store those return values.

So the current main loop cannot know whether all three channels confirmed TxDone.

The commented-out code in `main()` suggests that the author planned to use variables like:

```
ch37_ok
ch38_ok
ch39_ok
```

But those variables are not currently active.

The code is blocking

The firmware uses:

- `HAL_Delay()`
- `HAL_MAX_DELAY`

- polling loops
- `while(1)` error loops

This is okay for bring-up and simple beacon firmware.

But it means the CPU is not doing anything else while waiting.

No interrupts are used

DIO1 is configured as an input, but the code does not attach an interrupt handler.

Instead, it polls DIO1 after transmission.

Polling is simpler. Interrupts are more responsive but more complex.

Byte order matters

This firmware has several places where byte order matters:

- SX1280 register addresses are sent high byte first.
- BLE advertiser address byte order can look reversed depending on the view.
- Apple company ID is little-endian: `0x4C 0x00`.
- BLE access address is written as `0x8E 0x89 0xBE 0xD6`.

If a single byte order is wrong, the packet may not decode as expected.

`main.h` is small but critical

Most of the "behavior" is in `main.c`, but the pin map in `main.h` controls which physical pins are used.

If `main.h` does not match the PCB wiring, the firmware will appear broken even if `main.c` is logically correct.

30. Simplified Mental Model

You can think of the firmware as four layers.

```

Layer 4: Application behavior
    Send an iBeacon advertisement forever.

Layer 3: BLE packet construction
    Build ADV_NONCONN_IND packet with address, flags, iBeacon data.

Layer 2: SX1280 radio commands
    Set packet type, modulation, frequency, whitening, buffer, TX.

Layer 1: STM32 hardware control
    GPIO pins, SPI bus, clock setup, delays.
```

When debugging, move from bottom to top:

1. Are pins correct?
2. Does SPI work?
3. Does SX1280 status read correctly?

4. Do BLE registers read back correctly?
5. Does TxDone happen?
6. Does a BLE scanner decode the advertisement?

31. Complete Main Loop in One Sentence

After setup, the firmware repeatedly sends the same iBeacon-style BLE advertisement on BLE advertising channels 37, 38, and 39 using the SX1280 radio, with LED patterns used as the only visible debug output.

32. Practical Debug Checklist

If it does not work, check in this order:

1. Does the LED do anything at reset?
2. Does it slow-blink forever? If yes, focus on SPI/radio response.
3. Does it fast-blink forever? If yes, focus on BUSY pin/radio readiness.
4. Do you see the one alive blink? If yes, SPI status worked.
5. Do you see 9, 6, 3, 1? If yes, important BLE setup writes read back correctly.
6. Does DIO1 go high after TX? If not, focus on IRQ mapping and TX command.
7. Does a BLE scanner see the beacon? If not, focus on RF, packet bytes, whitening, antenna, and frequency.

33. Tiny Glossary

Term	Simple meaning
MCU	Microcontroller, the STM32 chip running the program
HAL	ST helper library for controlling STM32 hardware
GPIO	A digital input/output pin
SPI	A byte-based communication bus
NSS	SPI chip select pin
BUSY	SX1280 signal saying "do not send me a new command yet"
DIO1	SX1280 status/interrupt pin, used here for TxDone
IRQ	Interrupt request or event flag
Tx	transmit
Rx	receive
PDU	protocol data unit, basically a formatted packet
BLE advertising	broadcast packets that scanners can hear without connecting
iBeacon	Apple-style BLE beacon format inside manufacturer data
CRC	error-checking value
whitening	BLE bit scrambling process for RF reliability

34. The One-Layer-Deeper Technical Summary

The firmware programs the SX1280 with BLE packet type 0x04, BLE advertising access address 0x8E89BED6, CRC seed 0x555555, BLE modulation parameters, channel-specific RF frequencies, and channel-specific whitening seeds. It constructs a legacy non-connectable advertising PDU with a random advertiser address and iBeacon manufacturer data, writes the 38-byte PDU to TX buffer offset 0x80, starts transmission using SetTx, checks DIO1 for TxDone, clears IRQ flags, and repeats that process for BLE advertising channels 37, 38, and 39 forever.